

Chapitre 2 : Fichiers et erreurs, lire des données imparfaites

Par **Robert DIASSÉ**, Ingénieur Logiciel et Data

> **Cours** : Python pour la science des données · **Fil rouge** : les prix des denrées sur les marchés du Sénégal · **Version** : 2026-10-23

Fiche du chapitre

Objectifs	A la fin du chapitre, l'apprenant sait : 1) lire un fichier texte ligne par ligne avec `open` et `with` ; 2) lire et écrire un fichier CSV avec le module `csv`, en Python pur ; 3) convertir des valeurs lues (texte) en nombres et en dates ; 4) repérer et écarter les lignes invalides avec `try` / `except`, en gardant leur numéro et leur raison ; 5) faire le bilan des rejets ; 6) reconnaître et corriger un problème d'encodage ; 7) enregistrer et relire des résultats en JSON.
Prérequis	Chapitre 1 (notebooks, Colab) ; Python de L1 (listes, dictionnaires, boucles, fonctions)
Durée	🟢 Débutant : 7 h · 🔵 Standard : 5 h · 🟣 Expert : 3 h 30 (atelier compris)
Matériel	Google Colab ; les fichiers des relevés du PAM, des marchés et des relevés de terrain (fournis)
Livrable	Le notebook de l'atelier 2 : les 52 021 relevés chargés en Python pur et le bilan d'un fichier d'enquête abîmé

Comment lire ce chapitre

- **Noyau** : obligatoire pour tous les niveaux.
- 🟢 **Débutant** : explications pas à pas.
- 🔵 **Standard** : le niveau de référence.
- 🟣 **Expert** : approfondissements.

Ouvrez un nouveau notebook Colab, `chapitre2-fichiers`, et envoyez-y les trois fichiers fournis (icône 📁 à gauche, puis *Importer*) : les relevés de prix du PAM, la liste des marchés, et les relevés de terrain. Tapez les exemples dans l'ordre, une cellule par bloc de code.

---

1. Pourquoi lire les fichiers « à la main » ?

Au chapitre 4, pandas lira un fichier CSV en une ligne. Pourquoi alors apprendre à le faire en Python pur ? Parce que les données réelles sont **imparfaites** : une virgule à la place d'un point, une date au format français, une colonne oubliée, un prix « n.d. ». Quand pandas tombe sur ces problèmes, il les cache souvent (une colonne entière devient du texte) ou s'arrête avec un message difficile à comprendre. Savoir **lire ligne par ligne, convertir, et écarter proprement ce qui est faux**, c'est savoir ce qui se passe sous le capot, et c'est indispensable pour juger de la qualité de ses données.

Étape d'une analyse (chapitre 1)	Ce que ce chapitre apporte
Collecter	Ouvrir des fichiers texte, CSV, JSON
Nettoyer	Convertir, valider, écarter et compter les lignes invalides
Partager	Écrire des résultats en CSV et en JSON

2. Lire un fichier texte

Un fichier CSV est d'abord un **fichier texte**. Regardons ses trois premières lignes, telles quelles :

```
with open("wfp_food_prices_sen.csv", encoding="utf-8") as fichier:
    premieres = [fichier.readline() for _ in range(3)]
for ligne in premieres:
    print(repr(ligne))
```

```
'date,admin1,admin2,market,market_id,latitude,longitude,category,commodity,commodity_id,unit,price
flag,pricetype,currency,price,usdprice\n'
'2000-01-15,Dakar,Dakar,Tilene,447,14.68,-17.46,cereals and tubers,Maize
(imported),158,KG,actual,Retail,XOF,149,0.35\n'
'2000-01-15,Dakar,Dakar,Tilene,447,14.68,-17.46,cereals and
tubers,Millet,73,KG,actual,Retail,XOF,124.75,0.3\n'
```

Morceau	Ce qu'il veut dire
`open("...", encoding="utf-8")`	Ouvre le fichier en **lecture**, en précisant l'encodage (section 7)
`with ... as fichier:`	Le fichier est **refermé automatiquement** à la fin du bloc, même en cas d'erreur
`fichier.readline()`	Lit **une** ligne, avec son retour à la ligne `\n`
`[... for _ in range(3)]`	Une liste de 3 lignes ; `_` est un nom de variable qu'on n'utilise pas
`repr(ligne)`	Affiche le texte tel que Python le voit, avec les `\n` visibles

On voit la structure : une ligne d'**en-tête** (les noms des colonnes), puis une ligne par relevé, les valeurs séparées par des **virgules**. Comptons les lignes :

```
with open("wfp_food_prices_sen.csv", encoding="utf-8") as fichier:
    nombre = sum(1 for ligne in fichier)
print(nombre, "lignes, dont l'en-tête")
```

```
52022 lignes, dont l'en-tête
```

`for ligne in fichier` parcourt le fichier **ligne par ligne** sans tout charger en mémoire : `sum(1 for ...)` compte 1 par ligne.

3. Le module `csv`

Découper chaque ligne avec `ligne.split(",")` semble suffire. Mais si une valeur contient elle-même une virgule (un nom de marché, un prix `"387,5"`), elle est entourée de **guillemets**, et `split` la couperait en deux. Le module `csv` gère ces cas.

```
import csv

with open("wfp_food_prices_sen.csv", newline="", encoding="utf-8") as fichier:
    lecteur = csv.reader(fichier)
    entete = next(lecteur)
    premiere = next(lecteur)
print(entete)
print(premiere)
```

```
['date', 'admin1', 'admin2', 'market', 'market_id', 'latitude', 'longitude', 'category',
'commodity', 'commodity_id', 'unit', 'priceflag', 'pricetype', 'currency', 'price', 'usdprice']
['2000-01-15', 'Dakar', 'Dakar', 'Tilene', '447', '14.68', '-17.46', 'cereals and tubers', 'Maize
(imported)', '158', 'KG', 'actual', 'Retail', 'XOF', '149', '0.35']
```

Morceau	Ce qu'il veut dire
`newline=""`	Laisse le module `csv` gérer les fins de ligne (à mettre **toujours** avec `csv`)
`csv.reader(fichier)`	Un lecteur qui renvoie chaque ligne sous forme de **liste** de valeurs
`next(lecteur)`	La ligne suivante : la première fois, l'en-tête

Plus pratique : `csv.DictReader` utilise l'en-tête pour renvoyer chaque ligne sous forme de **dictionnaire**.

```
with open("wfp_food_prices_sen.csv", newline="", encoding="utf-8") as fichier:
    releves = list(csv.DictReader(fichier))
print(len(releves), "relevés")
print(releves[0]["commodity"], releves[0]["price"], type(releves[0]["price"]))
```

```
52021 relevés
Maize (imported) 149 <class 'str'>
```

Colonne	Ce qu'elle contient
`date`	La date du relevé (`AAAA-MM-JJ` , toujours le 15 du mois)
`admin1` , `admin2`	La région et le département
`market` , `latitude` , `longitude`	Le marché et sa position
`commodity` , `unit`	La denrée (en anglais) et l'unité (`KG` le plus souvent)
`price`	Le prix en F CFA (`currency` : `XOF`)

4. Tout est texte : convertir

Remarquez `<class 'str'>` : **toutes** les valeurs lues dans un CSV sont des **textes**, même les prix. Essayons de faire la somme des prix du riz importé :

```
riz = [r for r in releves if r["commodity"] == "Rice (imported)"]
try:
    print(sum(r["price"] for r in riz))
except TypeError as e:
    print("Erreur :", e)
```

```
Erreur : unsupported operand type(s) for +: 'int' and 'str'
```

`sum` commence à 0 (un `int`) et essaie d'y ajouter un texte : impossible. Il faut **convertir** :

```
prix_riz = [float(r["price"]) for r in riz]
print(len(prix_riz), "relevés de riz importé")
print(f"Prix moyen : {sum(prix_riz) / len(prix_riz):.1f} F CFA le kilo")
print("Du moins cher au plus cher :", min(prix_riz), "à", max(prix_riz))
```

```
9308 relevés de riz importé
Prix moyen : 310.7 F CFA le kilo
Du moins cher au plus cher : 150.0 à 600.0
```

Morceau	Ce qu'il veut dire
`[r for r in releves if ...]`	Une **liste en compréhension** filtrée : les relevés de riz importé seulement
`float(r["price"])`	Convertit le texte en nombre décimal
`:.1f`	Affiche avec une décimale

> 🟢 **Débutant : int ou float ?** `int("149")` donne 149, mais `int("124.75")` échoue (`ValueError: invalid literal for int() with base 10: '124.75'`). Pour des prix, des poids, des températures, utilisez `float` .

5. Vérifier avant de calculer

Avant de faire confiance à un calcul, on regarde les **valeurs extrêmes** et les catégories. Comptons les unités, et cherchons le prix le plus élevé :

```
unites = {}
for r in releves:
    unites[r["unit"]] = unites.get(r["unit"], 0) + 1
print(unites)
plus_cher = max(releves, key=lambda r: float(r["price"]))
print(plus_cher["commodity"], plus_cher["unit"], plus_cher["price"], plus_cher["market"],
        plus_cher["date"])
```

```
{'KG': 51854, 'Head': 167}
Livestock (cattle) Head 646500 Ndoumbé DIOP 2026-07-15
```

Morceau	Ce qu'il veut dire
`unites.get(cle, 0) + 1`	Le compteur de la clé, ou 0 s'il n'existe pas encore, plus 1
`max(releves, key=lambda r: float(r["price"]))`	Le relevé dont le prix (converti) est le plus grand. `key` dit **selon quoi** comparer ; `lambda r: ...` est une petite fonction écrite sur place

167 relevés ne sont pas au kilo mais **par tête** : ce sont des animaux (un bovin à 646 500 F CFA). Les mélanger aux prix au kilo fausserait toutes les moyennes. Règle d'or : **toujours vérifier les unités** avant de calculer.

6. Lire un fichier abîmé

Le fichier du PAM est propre : il a été vérifié avant d'être publié. Les fichiers saisis sur le terrain, eux, ne le sont pas. Le fichier fourni `releves_terrain.csv` imite un relevé saisi à la main par des enquêteurs : les prix viennent des vrais relevés de 2025, mais des **erreurs de saisie** typiques y ont été ajoutées volontairement.

```
with open("releves_terrain.csv", encoding="utf-8") as fichier:
    print(fichier.read())
```

```
date,region,marche,denree,unite,prix
2025-02-15,Kaolack,Kaolack,Rice (imported),KG,383.33
2025-04-15,Tambacounda,Bakel,Maize (local),KG,400
2025-07-15,Kaolack,Ndrame Escale,Groundnuts (unshelled),KG,"387,5"
2025-07-15,Kolda,Kolda,Millet,KG,300
2025-09-15,Dakar,Tilene,Beans (niebe),KG,
2025-12-15,Kedougou,Mako,Beans (niebe),KG,750
15/02/2025,Fatick,Diakhao,Rice (imported),KG,425
2025-12-15,Tambacounda,Mereto,Maize (local),KG,250
2025-03-15,Louga,Gouille Mbeuth,Groundnuts (unshelled),KG,-321.25
2025-12-15,Thies,Thilmakha,Rice (local),KG,425
2025-08-15,Kolda,Diaobe,Groundnuts (shelled),KG
2025-08-15,Saint Louis,Saint-Louis,Beans (niebe),KG,1150
```

Repérez à l'œil les cinq problèmes avant de continuer. Puis écrivons une fonction qui **convertit** une ligne, ou **lève une exception** si elle est invalide, et une boucle qui garde les bonnes lignes et **met de côté** les autres :

```python
from datetime import date


def convertir(ligne):
    if None in ligne.values():
        raise ValueError("colonne manquante")
    jour = date.fromisoformat(ligne["date"])
    prix = float(ligne["prix"])
    if prix <= 0:
        raise ValueError(f"prix négatif ou nul : {prix}")
    return {"date": jour, "marche": ligne["marche"], "denree": ligne["denree"], "prix": prix}


valides, rejets = [], []
with open("releves_terrain.csv", newline="", encoding="utf-8") as fichier:
    for numero, ligne in enumerate(csv.DictReader(fichier), start=2):
        try:
            valides.append(convertir(ligne))
        except ValueError as e:
            rejets.append({"ligne": numero, "raison": str(e)})

print(len(valides), "relevés valides,", len(rejets), "rejetés")
for r in rejets:
    print(f"  ligne {r['ligne']} : {r['raison']}")
```

```
7 relevés valides, 5 rejetés
  ligne 4 : could not convert string to float: '387,5'
  ligne 6 : could not convert string to float: ''
  ligne 8 : Invalid isoformat string: '15/02/2025'
  ligne 10 : prix négatif ou nul : -321.25
  ligne 12 : colonne manquante
```

Morceau	Ce qu'il veut dire
`if None in ligne.values():`	Si une ligne a moins de valeurs que l'en-tête, `DictReader` met `None` dans les colonnes manquantes
`date.fromisoformat(texte)`	Convertit `"2025-02-15"` en date ; lève une `ValueError` pour tout autre format
`raise ValueError(...)`	**Notre** règle : un prix doit être positif. On lève la même exception que Python, avec un message clair
`enumerate(..., start=2)`	Le numéro de la ligne **dans le fichier** : la ligne 1 est l'en-tête
`try` / `except ValueError as e`	Si `convertir` échoue, on garde le numéro et la raison (`str(e)`), et on **continue**

Cette méthode a trois qualités : le programme ne s'arrête jamais ; chaque rejet est **expliqué** ; et le numéro de ligne permet à l'enquêteur de retrouver et corriger sa saisie.

> 🔵 **Standard : corriger ou rejeter ?** `"387,5"` est sans doute 387,5 F CFA écrit avec une virgule ; `15/02/2025` est sans doute le 15 février. On **peut** les corriger automatiquement, c'est ce que vous ferez dans l'atelier. Mais une correction est une hypothèse : on la fait seulement quand elle est **sûre**, on la **compte**, et on le dit dans le bilan. Un prix manquant ou négatif, en revanche, ne se devine pas : il se rejette.

Le **bilan** des rejets fait partie du résultat d'une analyse :

```
taux = len(rejets) / (len(valides) + len(rejets))
print(f"Taux de rejet : {taux:.0%}")
```

```
Taux de rejet : 42%
```

Un taux de rejet de 42 % (ici, un petit fichier fabriqué exprès) dirait qu'il faut revoir la façon dont les données sont saisies, avant toute analyse.

7. L'encodage

Un fichier est une suite d'**octets** ; l'**encodage** dit comment les transformer en lettres. Les fichiers modernes sont en **UTF-8**. Mais un fichier enregistré par un vieux logiciel sous Windows (ou exporté par certaines versions d'Excel) est souvent en **cp1252**. Fabriquons-en un, avec un nom accentué, puis relisons-le :

```
with open("releves_terrain.csv", encoding="utf-8") as source:
    texte = source.read()
with open("releves_windows.csv", "w", encoding="cp1252") as copie:
    copie.write(texte.replace("Kedougou", "Kédougou"))

try:
    with open("releves_windows.csv", encoding="utf-8") as fichier:
        fichier.read()
except UnicodeDecodeError as e:
    print("Erreur :", e)
with open("releves_windows.csv", encoding="cp1252") as fichier:
    print([l for l in fichier if "Kédougou" in l])
```

```
Erreur : 'utf-8' codec can't decode byte 0xe9 in position 304: invalid continuation byte
['2025-12-15,Kédougou,Mako,Beans (niebe),KG,750\n']
```

Morceau	Ce qu'il veut dire
`open(..., "w", encoding="cp1252")`	Écrit un fichier (mode `"w"`, qui **efface** le contenu s'il existe) avec l'ancien encodage de Windows
`UnicodeDecodeError: 'utf-8' codec can't decode byte 0xe9`	« le décodeur UTF-8 ne sait pas lire l'octet 0xe9 » : c'est le « é » écrit en cp1252
`encoding="cp1252"` à la lecture	Le bon encodage : le « é » est retrouvé

Règles : **écrivez toujours en UTF-8** ; si un fichier reçu donne une `UnicodeDecodeError`, essayez `encoding="cp1252"`. Si les accents s'affichent `Ã©`, c'est l'inverse : un fichier UTF-8 lu en cp1252.

8. Le format JSON

8.1 Enregistrer des résultats

Le **JSON** enregistre des dictionnaires et des listes **avec leurs types** (un nombre reste un nombre). Calculons le prix moyen de quatre céréales en 2025 et enregistrons-le :

```
import json

moyennes = {}
for denree in ["Millet", "Rice (imported)", "Maize (local)", "Sorghum"]:
    prix = [float(r["price"]) for r in releves if r["commodity"] == denree and
         r["date"].startswith("2025")]
    moyennes[denree] = round(sum(prix) / len(prix), 1)

with open("moyennes_2025.json", "w", encoding="utf-8") as fichier:
    json.dump(moyennes, fichier, indent=2, ensure_ascii=False)
with open("moyennes_2025.json", encoding="utf-8") as fichier:
    print(fichier.read())
```

```
{
  "Millet": 304.0,
  "Rice (imported)": 384.4,
  "Maize (local)": 286.4,
  "Sorghum": 363.0
}
```

Morceau	Ce qu'il veut dire
`r["date"].startswith("2025")`	Le relevé date de 2025 (la date est encore un texte ici)
`json.dump(objet, fichier, ...)`	Écrit l'objet dans le fichier
`indent=2`	Présentation lisible, sur plusieurs lignes
`ensure_ascii=False`	Garde les accents tels quels

```
with open("moyennes_2025.json", encoding="utf-8") as fichier:
    relu = json.load(fichier)
print(relu["Millet"], type(relu["Millet"]))
print(relu == moyennes)
```

```
304.0 <class 'float'>
True
```

Contrairement au CSV, on relit **un nombre**, pas un texte : `relu` est identique à `moyennes`.

8.2 Des données imbriquées

JSON sait aussi enregistrer des structures **imbriquées**, comme un dictionnaire dont les valeurs sont des listes. Rangeons les marchés par région :

```
marches = {}
with open("wfp_markets_sen.csv", newline="", encoding="utf-8") as fichier:
    for m in csv.DictReader(fichier):
        marches.setdefault(m["admin1"], []).append(m["market"])
print(len(marches), "régions")
print(json.dumps({"Ziguinchor": marches["Ziguinchor"]}, ensure_ascii=False))
```

```
14 régions
{"Ziguinchor": ["Bignona", "Oussouye", "Zigiunchor", "Saint-Maur"]}
```

Morceau	Ce qu'il veut dire
`marches.setdefault(region, [])`	Renvoie la liste de la région, en la créant vide si elle n'existe pas encore
`.append(m["market"])`	Ajoute le marché à cette liste
`json.dumps(...)`	Comme `dump`, mais renvoie un **texte** au lieu d'écrire un fichier

Avez-vous remarqué `"Zigiunchor"` ? C'est une **vraie faute de frappe** dans le fichier officiel du PAM (pour Ziguinchor). Même les données publiées par une grande organisation contiennent des erreurs : il faut toujours regarder ses données.

> 🔵 **Standard : ce que JSON ne sait pas écrire** JSON connaît les textes, les nombres, `true` / `false`, `null`, les listes et les dictionnaires. Une **date** (objet `date`) provoque `TypeError: Object of type date is not JSON serializable` : il faut l'écrire en texte avec `jour.isoformat()` et la relire avec `date.fromisoformat(...)`.

9. Écrire un CSV de résultats

Enregistrons les relevés de terrain valides, pour les transmettre à un collègue :

```
with open("valides_terrain.csv", "w", newline="", encoding="utf-8") as fichier:
    ecrivain = csv.DictWriter(fichier, fieldnames=["date", "marche", "denree", "prix"])
    ecrivain.writeheader()
    for v in valides:
        ecrivain.writerow({**v, "date": v["date"].isoformat()})
with open("valides_terrain.csv", encoding="utf-8") as fichier:
    print(fichier.read())
```

```
date,marche,denree,prix
2025-02-15,Kaolack,Rice (imported),383.33
2025-04-15,Bakel,Maize (local),400.0
2025-07-15,Kolda,Millet,300.0
2025-12-15,Mako,Beans (niebe),750.0
2025-12-15,Mereto,Maize (local),250.0
2025-12-15,Thilmakha,Rice (local),425.0
2025-08-15,Saint-Louis,Beans (niebe),1150.0
```

Morceau	Ce qu'il veut dire
`csv.DictWriter(fichier, fieldnames=[...])`	Un écrivain qui écrit des dictionnaires, colonnes dans l'ordre donné
`writeheader()`	Écrit la ligne d'en-tête
`{**v, "date": v["date"].isoformat()}`	Une **copie** du dictionnaire, où seule la date est remplacée par son texte

> 🪛 **Essayez** : retirez `newline=""` et relancez la cellule, puis relisez le fichier avec `csv.reader`. Dans Colab (Linux), rien ne change ; sous Windows, une ligne vide apparaît après chaque ligne. Mettez-le toujours.

Exercices

Dans votre notebook du chapitre, à la suite des exemples :

- ★ **Exercice 1 : les températures.** Écrivez dans `temperatures_dakar.txt` une température par ligne : `27.5`, `28.1`, une ligne vide, `29.4`, `n.d.`, `30.2`, `28.8` (avec des espaces). Relisez le fichier : ignorez les lignes vides, convertissez les autres, comptez celles qui ne sont pas des nombres. Attendu : 5 valeurs, moyenne 28,8 °C, 1 valeur ignorée. (Sections 2 et 6.)
- ★ **Exercice 2 : les marchés par région.** Avec le fichier des marchés, comptez les marchés de chaque région et affichez les trois régions qui en ont le plus. Combien le fichier contient-il de marchés ? Comparez avec les 67 marchés qui ont des prix. (Sections 3 et 5.)
- ★★ **Exercice 3 : les extrémités du pays.** Quel est le marché le plus au **sud** (latitude la plus petite) ? Le plus à l'**est** (longitude la plus grande) ? Dans quelle région sont-ils ? Indice : `min(..., key=lambda m: float(m["latitude"]))`. (Section 5.)
- ★★ **Exercice 4 : la pluie.** Enregistrez en JSON ce dictionnaire de pluviométrie (valeurs **fictives**, en millimètres) : Dakar `{"juillet": 61, "aout": 168, "septembre": 132}`, Ziguinchor `{"juin": 120, "juillet": 340, "aout": 460, "septembre": 330}`, Matam `{"juillet": 95, "aout": "n.d.", "septembre": 70}`. Relisez-le et calculez le total de chaque station en ignorant les mois non numériques, que vous listerez. Attendu : 361, 1 250, 165 (avec août manquant). Indice : `isinstance(mm, (int, float))`. (Section 8.)
- ★★★ **Exercice 5 : des doublons ?** Un même relevé (même date, même marché, même denrée) apparaît-il deux fois dans le fichier du PAM ? Utilisez un `set` de tuples `(date, marche, denree)`. Combien de combinaisons différentes trouvez-vous ? Que concluez-vous ? (Sections 3 et 5.)

Atelier 2 : charger les relevés et faire le bilan des rejets

Le travail principal du chapitre : vous chargez en Python pur les 52 021 relevés du PAM, vous calculez l'évolution du prix du riz importé à Dakar année par année et l'enregistrez en JSON ; puis vous traitez un fichier d'enquête de 300 relevés saisis à la main, vous en faites le bilan des rejets par type d'erreur, vous corrigez ce qui peut l'être sans risque, et vous enregistrez le bilan. L'énoncé, le notebook de départ et le fichier d'enquête vous sont fournis séparément.

A retenir

- `with open(chemin, encoding="utf-8") as f:` ouvre et referme le fichier ; `for ligne in f` le lit ligne par ligne.
- `csv.reader` donne des listes, `csv.DictReader` des dictionnaires ; `newline=""` toujours. **Tout est texte** : convertir avec `float` (ou `int`).
- Avant de calculer : vérifier les **unités** et les **valeurs extrêmes**.
- Fichier abîmé : une fonction `convertir` qui lève une `ValueError` avec un message clair ; une boucle avec `try` / `except` ; une liste de rejets avec le **numéro de ligne** et la **raison** ; un **bilan** (taux de rejet).
- Corriger seulement ce qui est **sûr**, et le compter.
- Encodage : écrire en UTF-8 ; `UnicodeDecodeError` à la lecture : essayer `cp1252`.

- JSON : `json.dump` / `json.load` gardent les types ; les dates s'écrivent en texte (`isoformat`).

Erreurs fréquentes

Message ou symptôme	Ce que ça veut dire	Solution
`FileNotFoundError: [Errno 2] No such file or directory: '...'`	Fichier non envoyé dans Colab (ou effacé à la fin de la session), ou nom mal écrit	Renvoyer le fichier ; vérifier le nom exact dans le panneau 📁
`TypeError: unsupported operand type(s) for +: 'int' and 'str'`	Calcul sur des valeurs lues dans un CSV, restées en texte	`float(...)`
`ValueError: could not convert string to float: '387,5'`	Virgule décimale	`texte.replace(",", ".")` si c'est sûr ; sinon rejeter
`ValueError: could not convert string to float: ''`	Valeur vide	Rejeter (ou traiter comme manquante)
`ValueError: Invalid isoformat string: '15/02/2025'`	Date au format français	`datetime.strptime(texte, "%d/%m/%Y")` ou réordonner jour, mois, année
`TypeError: ... not 'NoneType'`	Colonne manquante sur cette ligne	Tester `None in ligne.values()`
`UnicodeDecodeError: 'utf-8' codec can't decode byte ...`	Fichier en cp1252	`encoding="cp1252"`
Moyenne absurde (des centaines de milliers de F)	Unités mélangées (le bétail par tête)	Filtrer sur l'unité
`TypeError: Object of type date is not JSON serializable`	Une date dans les données JSON	`.isoformat()`

Pour aller plus loin

- Le tutoriel officiel, « Lecture et écriture de fichiers » (en français) : https://docs.python.org/fr/3/tutorial/inputoutput.html#reading-and-writing-files
- Le module `csv` : https://docs.python.org/fr/3/library/csv.html
- Le module `json` : https://docs.python.org/fr/3/library/json.html
- Le jeu de données du PAM sur HDX, et sa description (en anglais) : https://data.humdata.org/dataset/wfp-food-prices-for-senegal